ASSISTED BY AI

Proyecto 1

Orquestación de inferencia en LLM

Grupo SI-35

Aroa Iriondo y Sergio Morales

1. Formulación del problema

1.1. Representación del estado del problema

Para modelar este problema de algoritmos de búsqueda, hemos definido un estado que captura toda la información necesaria para el desarrollo de la toma de decisiones en un punto determinado.

El estado está representado como un list con dos elementos fundamentales:

- Pos: Un vector numérico de dos coordenadas: c(fila,columna) que representa la posición espacial dentro de la cuadrícula.
- Mode: Una cadena de texto que indica el modo operativo en el que se encuentra en ese instante. Puede ser "GEN", si está realizando movimiento cardinales o el nombre de un proveedor específico de RAG ("vector DB_ECO", "vector DB_PRO").

Se utiliza una lista para almacenar tipos de datos de diferentes tipos. Era problemático incluir el Mode en la definición ya que el estado de búsqueda necesita todo el contexto de la acción anterior para calcular correctamente las latencias y añadir los costes de cambio de proveedor o switch lat. Si el estado solo contemplara pos, el algoritmo habría evaluado mal estados idénticos cuando llega a una casilla por un enlace RAG frente a llegar con movimientos cardinales.

La función initialize.problem(file) se encarga de devolver el problema que tiene todos los atributos necesarios para el correcto funcionamiento de los algoritmos. Problem tiene los siguientes atributos: name, state_initial, state_final, current_state, rag_names, actions_possible, map, params, providers. Los estados contienen dos argumentos, unas coordenadas(y, x) y el modo. Params contiene los argumentos gen_lat, switch_lat y eur_per_sec.

La comprobación está en la función is.final.state. Devuelve TRUE sólo si las coordenadas del estado actual state$mode coinciden con las del estado final final_state, lo que significa que has llegado a la meta.

1.2. Definición de las acciones

El vector de acciones permitidas actions_possible es dinámico y parametrizable según el entorno, contienen los 8 movimientos cardinales a los que se les añaden los saltos de RAG dinámicamente dependiendo de los rag links.

La función is.applicable() evalúa la viabilidad comprobando que el movimiento no se salga fuera del mapa y que la celda a la que vas no sea un obstáculo (#).

Si la acción es permitida, la función effect() computa y devuelve el nuevo estado. Calcula el respectivo movimiento, dependiendo si utiliza los movimientos cardinales o por el contrario aplica la teletransportación mediante los RAG.

La función get.cost() calcula el coste dependiendo del movimiento y el modo actual. Si el modo se mantiene en GEN se le aplica el coste de su latencia multiplicada por el coste por segundo, en cambio si el movimiento implica un cambio de modo (GEN a RAG ó 2 RAG distintos) se aplicaría el coste del proveedor + el coste de la latencia multiplicado por el coste por segundo + el switch_lat por el coste por segundo. Por otro lado, si el cambio es de un RAG a GEN, se cierra el ticket y por lo tanto el coste es el switch_lat* coste

por segundo + latencia de gen * coste por segundo. Por último si se mantiene en el mismo RAG el coste sería la latencia del rag por el coste por segundo.

2. Función de evaluación

Hemos implementado una función get.evaluation(), que su heurística principal es la distancia de Chebyshev. Dado que esta tiene la capacidad de moverse en 8 direcciones, es la más precisa a nivel de admisibilidad y medición en el tablero. Se estima calculando el valor absoluto del delta máximo entre la coordenada X y la coordenada Y (max(|x_actual - x_metal|, |y_actual - y_meta|).

Es admisible ya que calcula de forma teórica la cantidad mínima de saltos cardinales/diagonales para llegar al destino en los mapas dados.

3. Análisis de los resultados

3.1. Resultados Obtenidos

A continuación presentamos las tablas en las que se muestran los resultados de cada algoritmo para cada problema .

01-loop-trap

Name	Solution Found	End Reason	Actions	Cost	Expanded	Max Depth	Max Frontier	Run time	Final State
Breadth First Search	FALSE	Iterations	NA	NA	2500	4	25192	2.7265	?
Breadth First Search + GS	TRUE	Solution	5	6.0	84	5	25	0.0555	c(8,9), GEN
Depth First Search	FALSE	Iterations	NA	NA	2500	2499	13762	2.7770	?
Depth First Search + GS	TRUE	Solution	13	15.6	16	13	22	0.0072	c(8,9), GEN
Depth Limited Search - Limit=20	FALSE	Iterations	NA	NA	2500	20	140	0.1727	?
Depth Limited Search - Limit=20 + GS	TRUE	Solution	13	15.6	16	13	22	0.0099	c(8,9), GEN
Iterative Deepening Search - Final Limit=3	FALSE	Iterations	NA	NA	2500	3	41	0.1509	?
Iterative	TRUE	Solution	7	8.4	355	7	19	0.1464	c(8,9),

Deepening Search - Final Limit=8 + GS									GEN
Uniform Cost Search	FALSE	Iterations	NA	NA	2500	6	28852	55.3598	?
Uniform Cost Search + GS	TRUE	Solution	5	6.0	84	5	28	0.0600	c(8,9), GEN
Greedy Best-First Search	TRUE	Solution	5	6.0	6	5	52	0.0033	c(8,9), GEN
Greedy Best-First Search + GS	TRUE	Solution	5	6.0	6	5	25	0.0044	c(8,9), GEN
A* Search	TRUE	Solution	5	6.0	20	5	186	0.0141	c(8,9), GEN
A* Search + GS	TRUE	Solution	5	6.0	11	5	28	0.0083	c(8,9), GEN

02-rag-dilema

Name	Solution Found	End Reason	Actions	Cost	Expanded	Max Depth	Max Frontier	Run time	Final State
Breadth First Search	TRUE	Solution	3	5.4	21	3	41	0.0903	c(11,12), GEN
Breadth First Search + GS	TRUE	Solution	3	5.4	13	3	11	0.0049	c(11,12), GEN
Depth First Search	FALSE	Iteration s	NA	NA	2500	2499	5000	0.8251	?
Depth First Search + GS	TRUE	Solution	19	22.8	56	37	7	0.0157	c(11,12), GEN
Depth Limited Search - Limit=20	TRUE	Solution	20	25.8	760	20	40	0.0613	c(11,12), GEN
Depth Limited Search - Limit=20 + GS	TRUE	Solution	19	22.8	31	20	5	0.0062	c(11,12), GEN
Iterative Deepening Search - Final Limit=4	TRUE	Solution	3	5.4	33	3	7	0.0065	c(11,12), GEN
Iterative Deepening Search - Final Limit=4 + GS	TRUE	Solution	3	5.4	23	3	4	0.0146	c(11,12), GEN
Uniform Cost Search	TRUE	Solution	3	5.4	31	4	59	0.0082	c(11,12), GEN

Uniform Cost Search + GS	TRUE	Solution	3	5.4	15	4	11	0.0043	c(11,12), GEN
Greedy Best-First Search	TRUE	Solution	3	5.4	4	3	6	0.0012	c(11,12), GEN
Greedy Best-First Search + GS	TRUE	Solution	3	5.4	4	3	5	0.0027	c(11,12), GEN
A* Search	TRUE	Solution	3	5.4	5	3	10	0.0018	c(11,12), GEN
A* Search + GS	TRUE	Solution	3	5.4	5	3	6	0.0025	c(11,12), GEN

03-deep-corridor

Name	Solution Found	End Reason	Actions	Cost	Expanded	Max Depth	Max Frontier	Run time	Final State
Breadth First Search	FALSE	Iterations	NA	NA	2500	7	6076	0.9453	?
Breadth First Search + GS	TRUE	Solution	12	15.65	57	12	8	0.0245	c(11,12), GEN
Depth First Search	FALSE	Iterations	NA	NA	2500	2499	5001	0.7681	?
Depth First Search + GS	TRUE	Solution	31	37.20	39	31	6	0.0080	c(11,12), GEN
Depth Limited Search - Limit=20	FALSE	Iterations	NA	NA	2500	20	49	0.1905	?
Depth Limited Search - Limit=20 + GS	TRUE	Solution	12	15.65	44	20	5	0.0156	c(11,12), GEN
Iterative Deepening Search - Final Limit=6	FALSE	Iterations	NA	NA	2500	6	19	0.1926	?
Iterative Deepening Search - Final Limit=13 + GS	TRUE	Solution	12	15.65	327	12	5	0.1495	c(11,12), GEN
Uniform Cost Search	FALSE	Iterations	NA	NA	2500	7	6196	11.1364	?
Uniform Cost Search	TRUE	Solution	12	15.65	57	12	8	0.0252	c(11,12), GEN

+ GS									
Greedy Best-First Search	FALSE	Iterations	NA	NA	2500	32	4251	7.0278	?
Greedy Best-First Search + GS	TRUE	Solution	31	37.20	46	31	9	0.0116	c(11,12), GEN
A* Search	TRUE	Solution	12	15.65	267	12	594	0.1685	c(11,12), GEN
A* Search + GS	TRUE	Solution	12	15.65	26	12	9	0.0073	c(11,12), GEN

o4-cost-trap

Name	Solution Found	End Reason	Actions	Cost	Expanded	Max Depth	Max Frontier	Run time	Final State
Breadth First Search	TRUE	Solution	1	2.7	3	1	3	0.0861	c(10,10), VectorDB_PRO
Breadth First Search + GS	TRUE	Solution	1	2.7	3	1	2	0.0042	c(10,10), VectorDB_PRO
Depth First Search	FALSE	Iterations	NA	NA	2500	2499	3749	0.6986	?
Depth First Search + GS	TRUE	Solution	17	20.4	19	17	4	0.0040	c(10, 10),GEN
Depth Limited Search - Limit=20	TRUE	Solution	20	24.0	1487	20	29	0.1255	c(10, 10),GEN
Depth Limited Search - Limit=20 + GS	TRUE	Solution	17	20.4	19	17	4	0.0041	c(10, 10),GEN
Iterative Deepening Search - Final Limit=2	TRUE	Solution	1	2.7	4	1	2	0.0028	c(10,10), VectorDB_PRO
Iterative Deepening Search - Final Limit=2 + GS	TRUE	Solution	1	2.7	4	1	2	0.0028	c(10,10), VectorDB_PRO
Uniform Cost Search	TRUE	Solution	1	2.7	5	2	7	0.0020	c(10,10), VectorDB_PRO
Uniform Cost Search	TRUE	Solution	1	2.7	4	2	3	0.0020	c(10,10), VectorDB_PRO

+ GS									
Greedy Best-First Search	TRUE	Solution	1	2.7	2	1	2	0.0008	c(10,10), VectorDB_PRO
Greedy Best-First Search + GS	TRUE	Solution	1	2.7	2	1	2	0.0007	c(10,10), VectorDB_PRO
A* Search	TRUE	Solution	1	2.7	2	1	2	0.0008	c(10,10), VectorDB_PRO
A* Search + GS	TRUE	Solution	1	2.7	2	1	2	0.0008	c(10,10), VectorDB_PRO

05-greedy-trap

Name	Solution Found	End Reason	Actions	Cost	Expanded	Max Depth	Max Frontier	Run time	Final State
Breadth First Search	FALSE	Iterations	NA	NA	2500	5	15514	1.5199	?
Breadth First Search + GS	TRUE	Solution	11	15.0	104	11	19	0.0374	c(5,12), GEN
Depth First Search	FALSE	Iterations	NA	NA	2500	2499	8753	1.4716	?
Depth First Search + GS	TRUE	Solution	15	19.8	17	15	20	0.0067	c(5,12), GEN
Depth Limited Search - Limit=20	FALSE	Iterations	NA	NA	2500	20	80	0.1282	?
Depth Limited Search - Limit=20 + GS	TRUE	Solution	15	19.8	17	15	20	0.0054	c(5,12), GEN
Iterative Deepening Search - Final Limit=5	FALSE	Iterations	NA	NA	2500	5	25	0.1174	?
Iterative Deepening Search - Final Limit=16 + GS	TRUE	Solution	15	19.8	916	15	20	0.2561	c(5,12), GEN
Uniform Cost Search	FALSE	Iterations	NA	NA	2500	5	15516	28.6506	?
Uniform Cost Search + GS	TRUE	Solution	12	14.4	104	12	18	0.0532	c(5,12), GEN
Greedy Best-First Search	TRUE	Solution	16	19.2	197	16	793	0.1386	c(5,12), GEN
Greedy	TRUE	Solution	16	19.2	23	16	36	0.0110	c(5,12), GEN

Best-First Search + GS									
A* Search	FALSE	Iterations	NA	NA	2500	6	13477	27.8905	?
A* Search + GS	TRUE	Solution	12	14.4	64	12	29	0.0272	c(5,12), GEN

o6-shallow-illusion

Name	Solution Found	End Reason	Actions	Cost	Expanded	Max Depth	Max Frontier	Run time	Final State
Breadth First Search	TRUE	Solution	2	2.4	4	2	8	0.0022	c(2,2), GEN
Breadth First Search + GS	TRUE	Solution	2	2.4	4	2	4	0.0023	c(2,2), GEN
Depth First Search	TRUE	Solution	2	2.4	3	2	4	0.0018	c(2,2), GEN
Depth First Search + GS	TRUE	Solution	2	2.4	3	2	2	0.0038	c(2,2), GEN
Depth Limited Search - Limit=20	TRUE	Solution	2	2.4	3	2	4	0.0021	c(2,2), GEN
Depth Limited Search - Limit=20 + GS	TRUE	Solution	2	2.4	3	2	2	0.0014	c(2,2), GEN
Iterative Deepening Search - Final Limit=3	TRUE	Solution	2	2.4	7	2	4	0.0054	c(2,2), GEN
Iterative Deepening Search - Final Limit=3 + GS	TRUE	Solution	2	2.4	7	2	2	0.0062	c(2,2), GEN

Uniform Cost Search	TRUE	Solution	2	2.4	4	2	8	0.0014	c(2,2), GEN
Uniform Cost Search + GS	TRUE	Solution	2	2.4	4	2	4	0.0036	c(2,2), GEN
Greedy Best-First Search	TRUE	Solution	2	2.4	3	2	4	0.0014	c(2,2), GEN
Greedy Best-First Search + GS	TRUE	Solution	2	2.4	3	2	2	0.0015	c(2,2), GEN
A* Search	TRUE	Solution	2	2.4	3	2	4	0.0012	c(2,2), GEN
A* Search + GS	TRUE	Solution	2	2.4	3	2	2	0.0013	c(2,2), GEN

3.2 Comparación de Resultados

Analizando estos resultados hemos llegado a las siguientes conclusiones:

- Blind Search vs Informed Search: El uso de las heurísticas (greedy y A*), demostró reducir el uso de la memoria (frontier size) limitando las expansiones frente a BFS y permitiendo encontrar el resultado final en milisegundos.
- GS(Graph Search): Observando la tabla general, resulta obvio que aplicar control de estados repetidos es obligatorio en problemas espaciales para evitar entrar en bucles de ida y vuelta.
- Optimalidad(BFS vs UCS/A*): Mientras que Breadth-First es óptimo si evaluamos la cantidad de variables o pasos. Por lo tanto BFS no halla la ruta más económica. Por otro lado, Uniform cost Search y A* garantizan una optimalidad rigurosa a nivel financiero.
- Complejidad/Completitud: Algoritmos como DFS son de los más rápidos si tienen la suerte de atinar pero caen en faltas gravísimas de profundidad. La búsqueda más balanceada recae sobre el A* Search con Graph Search.

La estrategia más adecuada es A*Search acoplado a Graph Search(control de ciclos). Garantiza la completitud del problema, evaluar financieramente la ecuación logrando optimalidad económica y sufre menos cuellos de botella RAM dado que la heurística encamina rápido su avance cortando la expansión a comparación de un Uniform Cost o un Breadth First.

4. Declaración de uso de herramientas de IA

Para este proyecto hemos utilizado principalmente chatGPT para identificar errores de sintaxis de R, aprender sobre la lectura de ficheros y la explicación de bloques de código para entender la plantilla que se nos proporcionaba.

5. Bibliografía

- Material didáctico del Alud
- Documentación de R
- Devpath